Big Data and NoSQL

Lab 5 – Distributed MongoDB: Replication and Sharding

Hajer Akid
Associate Professor – Université Côte d'Azur
Master 2 – SLCD

13 October 2026

Instructions

Create a document named `Lab5_Firstname_Lastname` using an editor of your choice. Use this document throughout the lab to record the requested observations and to answer all questions labelled **Q**.

File to submit: `Lab5_Firstname_Lastname`

Required datasets: `movielens_movies.json` and `movielens_users.json`. Download both files from Moodle and place them directly inside your `Lab5` working directory.

Important. MongoDB Community Server was installed during a previous session. We will run several local MongoDB processes on different ports in order to simulate several nodes of a distributed system on one physical machine.

The lab is designed to work on both **macOS** and **Windows**. Unless a command is explicitly labelled macOS or Windows, the same command can be used on both systems. Long MongoDB commands are written on a single line so that they can be copied directly into either Terminal on macOS or PowerShell on Windows.

Run commands such as `mongod`, `mongos`, `mongosh`, and `mongoimport` in a normal terminal. Run commands such as `rs.status()`, `sh.status()`, and `db.users.findOne()` inside `mongosh`. Keep the terminals used to run `mongod` and `mongos` open. Naming the terminals (for example `RS-1`, `RS-2`, `CONFIG`, `SHARD-1`) will make the lab easier to follow.

Objectives

The goal of this lab is to observe, on a concrete MongoDB deployment, the two mechanisms studied in the lecture:

- **replication**: several nodes maintain copies of the same logical data in order to improve availability and fault tolerance;
- **sharding**: different parts of a collection are distributed across several shards in order to distribute storage and workload.

By the end of the lab, you should be able to identify primary and secondary members of a replica set, observe automatic replication and election after a failure, build a small sharded cluster, shard the MovieLens `users` collection using a hashed shard key, inspect its physical distribution, and distinguish a targeted query from a query involving several shards.

Tooling. We use MongoDB Community Server, `mongosh`, and `mongoimport`. If `mongosh` is not available in your terminal, you may use the shell embedded in MongoDB Compass. If `mongoimport` is not available, you may use MongoDB Compass to import the JSON file, but you must connect Compass to the correct port indicated in the corresponding task.

Part 1 – Preparing the local environment

Each `mongod` process needs its own storage directory and its own network port. We first verify the required executables and prepare the working directory.

Tasks

T1. In a terminal, verify the commands that will be used in this lab:

```
mongod --version
mongos --version
mongosh --version
mongoimport --version
```

If `mongosh` is unavailable, use the shell embedded in MongoDB Compass for shell commands. If `mongoimport` is unavailable, use the Compass import procedure described later in the lab.

T2. Create a working directory named `Lab5`. Move `movielens_movies.json` and `movielens_users.json` into it, then create the directories needed for the replica set.

macOS – Terminal

```
mkdir -p Lab5/data/rs1 Lab5/data/rs2 Lab5/data/rs3
cd Lab5
```

Windows – PowerShell

```
New-Item -ItemType Directory -Force Lab5\data\rs1,Lab5\data\rs2,Lab5\data\rs3
Set-Location Lab5
```

The directory should now contain:

```
Lab5/
  movielens_movies.json
  movielens_users.json
  data/
    rs1/
    rs2/
    rs3/
```

All relative paths below assume that your terminal is positioned inside Lab5. Whenever you open a new terminal for a command involving `data/...` or a JSON file, first navigate to your Lab5 folder.

T3. Before starting MongoDB processes, check that the ports used in this lab are not already occupied: 27117, 27118, 27119, 27200, 27301, 27302, 27303, and 27400.

macOS – example for port 27117

```
lsof -nP -iTCP:27117 -sTCP:LISTEN
```

Windows – example for port 27117

```
netstat -ano | findstr :27117
```

No output means that the port is free. If a port is already occupied, ask the instructor before changing the port plan.

Question

Q1. Why must the three MongoDB server processes use different `dbPath` values and different port numbers even though they run on the same physical machine?

Part 2 – Building a three-member replica set

A replica set contains several MongoDB servers that maintain copies of the same logical data. One member is elected **primary**; the other members normally act as **secondaries**. We use three data-bearing voting members so that a majority can still exist if one node fails.

Tasks

T4. Open **three terminals**. In each one, first make sure that you are inside the `Lab5` directory. Start one MongoDB server in each terminal and do not close these terminals.

Terminal RS-1

```
mongod --replSet rsLab --dbpath data/rs1 --port 27117 --bind_ip localhost
```

Terminal RS-2

```
mongod --replSet rsLab --dbpath data/rs2 --port 27118 --bind_ip localhost
```

Terminal RS-3

```
mongod --replSet rsLab --dbpath data/rs3 --port 27119 --bind_ip localhost
```

T5. Open a fourth terminal and connect to the first server:

```
mongosh --port 27117
```

Initialize the replica set:

```
rs.initiate({
  _id: "rsLab",
  members: [
    {_id: 0, host: "localhost:27117"},
    {_id: 1, host: "localhost:27118"},
    {_id: 2, host: "localhost:27119"}
  ]
})
```

Wait a few seconds for the election to complete.

T6. Inspect the runtime state:

```
rs.status()
```

For a shorter view:

```
rs.status().members.map(m => ({name: m.name, state: m.stateStr}))
```

Record which port is currently `PRIMARY` and which ports are `SECONDARY`.

T7. Inspect the configured membership:

```
rs.conf()
```

Compare `rs.conf()` with `rs.status()`: one describes the configured members; the other describes the current runtime state.

Questions

Q2. What is the role of the primary member? What is the role of a secondary member?

Q3. Do the three members contain different fragments of the database, or copies of the same logical data? Which mechanism from the lecture does this illustrate?

Part 3 – Importing MovieLens and observing replication

As in the original MovieLens exercise, we use both supplied datasets. The files `movielens_movies.json` and `movielens_users.json` are imported once through the primary. We then verify that both collections are automatically replicated to the secondaries.

Tasks

T8. Verify again which member is currently `PRIMARY`. The following commands assume that `27117` is primary; replace the port if necessary.

Open a **new normal terminal** (not a `mongosh` prompt). Navigate to the `Lab5` directory that contains the JSON files. For example, on macOS, if you created it in your home directory:

```
cd ~/Lab5
ls
```

On Windows, use `Set-Location` followed by the actual location of your Lab5 folder (for example `Set-Location Lab5` when starting in its parent directory); check with `Get-ChildItem`. Then import both datasets:

```
mongoimport --host localhost --port 27117 --db TDLE --collection movies --file movielens_movies.json
mongoimport --host localhost --port 27117 --db TDLE --collection users --file movielens_users.json
```

If you repeat an import: error `E11000` generally means that document IDs already exist. To restart an import from scratch, add the option `--drop` to that `mongoimport` command. **Warning:** this deletes the target collection and its indexes before importing; use it only for the unsharded collections in this step, when no work needs to be preserved.

Compass fallback if `mongoimport` is unavailable. Connect Compass to `mongodb://localhost:27117`, create/open database `TDLE`, then import `movielens_movies.json` into collection `movies` and `movielens_users.json` into collection `users` using **Add Data** > **Import JSON or CSV**. The connection must be made to the current primary.

T9. In a new normal terminal, connect to the current primary (change port if necessary) and verify both imports:

```
mongosh --port 27117
```

Then run:

```
use TDLE
db.movies.countDocuments()
db.users.countDocuments()
db.movies.findOne({_id: 260})
db.users.findOne({_id: 6040})
```

The movie with `_id: 260` should be *Star Wars: Episode IV - A New Hope (1977)*. The user with `_id: 6040` is also a convenient reference document for later checks.

T10. Choose a member whose state is `SECONDARY`. Keep the primary connection open. In a **new normal terminal**, open a direct connection to that secondary. For example:

```
mongosh "mongodb://localhost:27118/?directConnection=true"
```

If `27118` is not secondary, use another secondary port. Allow reads on this direct secondary connection:

```
db.getMongo().setReadPref("secondary")
use TDLE
db.movies.countDocuments()
db.users.countDocuments()
db.movies.findOne({_id: 260})
db.users.findOne({_id: 6040})
```

Compare the results with those obtained from the primary.

T11. While still connected directly to the secondary, try to insert a movie document:

```
db.movies.insertOne({
  _id: 5000,
  title: "Lab 5 Replication Test (2026)",
  genres: "Test"
})
```

Record the result or error message. This failure is expected.

T12. Return to the primary and insert the same test document there:

```
use TDLE
db.movies.insertOne({
  _id: 5000,
  title: "Lab 5 Replication Test (2026)",
  genres: "Test"
})
```

Then return to the secondary and verify that it becomes visible:

```
db.movies.findOne({_id: 5000})
```

Questions

Q4. Why are both MovieLens collections visible on a secondary even though each file was imported only once through the primary?

Q5. Why is the insert rejected when you connect directly to a secondary?

Q6. MongoDB replica-set replication is normally asynchronous. What consequence can this have for a read performed on a secondary immediately after a write on the primary?

Part 4 – Testing fault tolerance and election

We now deliberately stop nodes. The objective is to observe which failures a three-member replica set can tolerate and how a new primary is elected.

Tasks

T13. On the current primary, insert a marker document:

```
use TDLE
db.movies.insertOne({
  _id: 5001,
  title: "FAILOVER_TEST",
  genres: "Test"
})
```

T14. Stop **one secondary** with `Ctrl+C` in its `mongod` terminal. From the primary, inspect the status:

```
rs.status().members.map(m => ({name: m.name, state: m.stateStr, health: m.health}))
```

Verify whether the primary remains available. Then restart the stopped secondary using exactly the same `mongod` command and the same `dbPath`. Wait until it returns to state `SECONDARY`.

T15. Now stop the current **primary** with `Ctrl+C`. Wait approximately 5–15 seconds. Connect to one of the two remaining members and inspect `rs.status()`. Identify the newly elected primary.

T16. Connect to the new primary and verify that the marker document survived the failure:

```
use TDLE
db.movies.findOne({_id: 5001})
```

Then insert another marker:

```
db.movies.insertOne({
  _id: 5002,
  title: "AFTER_ELECTION",
  genres: "Test"
})
```

T17. Restart the old primary with its original `mongod` command. After a few seconds, inspect `rs.status()` again. Record the state of the restarted member and verify that document `_id: 5002` eventually appears there.

Questions

Q7. Why can the two remaining members elect a new primary after one of the three voting members fails? Relate your answer to the notion of a majority.

Q8. Suppose only one of the three voting members remains reachable. Can that member elect itself primary and continue accepting writes? Explain.

Q9. Does replication reduce the amount of physical storage required by the system? Explain the trade-off between storage cost and availability.

Part 5 – Building a small sharded cluster

Stop the three `mongod` processes from Parts 2–4 with Ctrl+C in their **server terminals** before continuing. Do not simply close the `mongosh` clients. We now build a **separate** deployment for sharding. Do not reuse the replica-set data directories.

A MongoDB sharded cluster contains:

- **shards**, which store application data;
- **config servers**, which store cluster metadata;
- **mongos**, which is the query router and the entry point used by clients.

For this lab, the config server and each shard use a single-member replica set to keep the number of local processes manageable. This is a pedagogical simplification, not a production architecture.

Tasks

T18. Create the new directories.

macOS – Terminal

```
mkdir -p data/config data/shard1 data/shard2 data/shard3
```

Windows – PowerShell

```
New-Item -ItemType Directory -Force data\config,data\shard1,data\shard2,data\shard3
```

T19. Start the **config server** in a terminal:

```
mongod --configsvr --replSet configRS --dbpath data/config --port 27200 --bind_ip localhost
```

Connect to it from another terminal:

```
mongosh --port 27200
```

Initialize the config-server replica set:

```
rs.initiate({
  _id: "configRS",
  configsvr: true,
  members: [{_id: 0, host: "localhost:27200"}]
})
```

Wait until this member becomes `PRIMARY`.

T20. Start two shard servers.

Terminal SHARD-1

```
mongod --shardsvr --replSet shard1RS --dbpath data/shard1 --port 27301 --bind_ip localhost
```

Terminal SHARD-2

```
mongod --shardsvr --replSet shard2RS --dbpath data/shard2 --port 27302 --bind_ip localhost
```

Initialize each shard replica set separately: In a normal terminal:

```
mongosh --port 27301
```

Then, inside that mongosh session:

```
rs.initiate({_id: "shard1RS", members: [{_id: 0, host: "localhost:27301"}]})
```

In another normal terminal:

```
mongosh --port 27302
```

Then, inside that mongosh session:

```
rs.initiate({_id: "shard2RS", members: [{_id: 0, host: "localhost:27302"}]})
```

Wait until each member is `PRIMARY` in its own replica set.

T21. Start the **mongos router**:

```
mongos --configdb configRS/localhost:27200 --port 27400 --bind_ip localhost
```

From now on, interact with the sharded cluster through this router:

```
mongosh --port 27400
```

T22. In the shell connected to `mongos`, add the two shards:

```
sh.addShard("shard1RS/localhost:27301")
sh.addShard("shard2RS/localhost:27302")
sh.status()
```

Verify that both shards appear in the cluster status before continuing.

Questions

Q10. What information is stored by the config server, and what is the role of `mongos`?

Q11. Why should an application connect to `mongos` instead of choosing a shard directly?

Part 6 – Sharding the MovieLens users collection

We now follow the same data choice as the original MovieLens exercise: the `users` collection is sharded using a **hashed shard key on `age`**. This is chosen for learning purposes: age has low cardinality and would not generally be an ideal shard key for a production collection. This part is separate from the replica-set deployment: documents in one logical collection will now be distributed between different shards.

Tasks

T23. In the shell connected to `mongos` on port `27400`, create the database and the empty `users` collection:

```
use TDLE
db.createCollection("users")
```

Enable sharding for the database and create the hashed shard-key index:

```
sh.enableSharding("TDLE")
db.users.createIndex({age: "hashed"})
```

Shard the collection:

```
sh.shardCollection("TDLE.users", {age: "hashed"})
```

Inspect the cluster before loading the data:

```
sh.status()
```

T24. Open a **new normal terminal**, navigate to the `Lab5` working directory containing the JSON files (e.g., `cd /Lab5` on macOS if located in your home directory), and import `movielens_users.json`, this time through **mongos** (port 27400), not directly through a shard:

```
mongoimport --host localhost --port 27400 --db TDLE --collection users --file movielens_users.json
```

Compass fallback if `mongoimport` is unavailable. Connect Compass to `mongodb://localhost:27400`, open database `TDLE` and collection `users`, then import `movielens_users.json`. It is essential to connect Compass to port `27400`, because this is the `mongos` router.

T25. Back in the shell connected to `mongos`, verify the logical collection:

```
use TDLE
db.users.countDocuments()
db.users.findOne({_id: 6040})
```

Then inspect the physical distribution:

```
db.users.getShardDistribution()
sh.status()
```

Record approximately how many documents are stored on each shard. The numbers do not have to be exactly equal.

T26. Inspect the per-shard distribution **through mongos**, rather than issuing normal application

queries directly to individual shards (which are restricted in MongoDB 8.x). In the terminal connected to `mongos`, run:

```
use TDLE
db.users.getShardDistribution()
db.users.countDocuments()
```

Record the estimated per-shard counts and compare them with the total count through `mongos`. These figures may be approximate or temporarily differ during migrations.

T27. Compare two query-routing situations through `mongos`.

Query A – equality on the hashed shard key (execute through mongos)

```
db.users.find({age: 32}).explain("executionStats")
```

Query B – range on the same logical field

```
db.users.find({age: {$gte: 25, $lte: 35}}).explain("executionStats")
```

You do not need to analyse every field of the explain output. Focus on how many shards are involved in each query.

Questions

Q12. In Parts 2–4, each replica-set member had copies of both MovieLens collections. In Part 6, each shard contains only part of the `users` collection. Explain the conceptual difference.

Q13. Why can an equality query such as `age = 32` be routed using the hashed shard key?

Q14. Why is a range query such as `25 <= age <= 35` less naturally suited to hashed sharding? Relate your answer to the loss of ordering after hashing.

Q15. The document counts on the two shards may be different. Does this mean that some documents were lost? How can you verify your answer?

Part 7 – Adding a new shard and observing rebalancing

A distributed system should be able to evolve when capacity changes. We now add a third shard and observe how the cluster distribution changes.

Tasks

T28. In a new terminal positioned inside Lab5, start the third shard:

```
mongod --shardsvr --replSet shard3RS --dbpath data/shard3 --port 27303 --bind_ip localhost
```

Initialize it: In another normal terminal:

```
mongosh --port 27303
```

Then, inside that mongosh session:

```
rs.initiate({_id: "shard3RS", members: [{_id: 0, host: "localhost:27303"}]})
```

Wait until it becomes `PRIMARY`.

T29. Return to the shell connected to `mongos` and add the new shard:

```
sh.addShard("shard3RS/localhost:27303")
sh.status()
```

T30. Inspect the distribution again:

```
use TDLE
db.users.getShardDistribution()
```

MongoDB may need time to move data ranges between shards. Wait briefly and run the command again. With this small teaching dataset, adding a shard **may produce no visible data movement**; that is not a failure. Record whether the third shard appears and whether the distribution changes. The important point is to understand the balancer and why migrations may be necessary in larger deployments.

Questions

Q16. Why can adding a shard require data movement even though no application document was modified?

Q17. What is the purpose of rebalancing in a sharded cluster?

Part 8 – Final synthesis

In a production MongoDB deployment, a shard is commonly implemented as a replica set. The two mechanisms therefore coexist: sharding distributes different parts of the data; replication creates redundant copies inside each shard.

Questions

Q18. Consider a deployment with three shards, where each shard is a three-member replica set. How many data-bearing MongoDB nodes are used for the shards in total? Which mechanism provides horizontal distribution, and which mechanism provides redundancy?

Q19. Why is it incorrect to describe fragmentation/sharding and replication as opposite techniques? Give a short architecture example that combines both.

Takeaway

Replication and sharding solve different problems. A replica set maintains copies of the same logical data to improve availability and fault tolerance. Sharding distributes different parts of a collection across shards to scale storage and workload. MongoDB commonly combines both mechanisms in the same distributed architecture.